



This is Part-04 DevOps Scenario Based Interview Questions and Answers

Scenario: You deployed a new application version in Kubernetes and it's crashing. How do you debug?

Here's a **ready-to-use, step-by-step Kubernetes pod debugging checklist** you can follow every time a pod crashes. You can even keep this as a reference or printout.

Kubernetes Pod Crash Debugging Checklist

1. Check Pod Status

```
kubectl get pods
```

- ✓ Note the **STATUS**:
 - `CrashLoopBackOff` → Pod is repeatedly failing.
 - `Error` → Pod exited with a non-zero code.
 - `Pending` → Might be scheduling issue.

2. Describe the Pod

```
kubectl describe pod <pod-name>
```

- ✓ Look at **Events** section for:
 - Image pull errors (`ErrImagePull`, `ImagePullBackOff`)
 - Port conflicts
 - Resource limits exceeded
 - Missing Secrets or ConfigMaps
- ✓ Check **Container State** for exit codes.

3. Inspect Logs

```
kubectl logs <pod-name>
```

- ✓ For crashed pods:

```
kubectl logs <pod-name> --previous
```

- Look for:
 - Exceptions / stack traces
 - Misconfigurations / missing files

- Failed connections / dependencies

4. Verify Deployment or ReplicaSet

```
kubectl get deployment <deployment-name>
kubectl describe deployment <deployment-name>
```

- ✓ Check:
 - Correct **image version**
 - Environment variables
 - Volume mounts
 - Resource limits (CPU/memory)

5. Test in Debug Mode (Optional)

```
kubectl run -it --rm debug-pod --image=<image-name> -- /bin/sh
```

- ✓ Test:
 - File paths
 - Environment variables
 - Connectivity to other services
 - Dependencies

6. Check Cluster-Level Issues

- Nodes:

```
kubectl describe node <node-name>
kubectl top node
```

- Resources: CPU, memory, disk
- Network policies or firewall issues
- Persistent Volumes, Secrets, ConfigMaps

7. Check Events Chronologically

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

- ✓ Look for failures or warnings around pod creation time.

8. Verify Rollout Status

```
kubectl rollout status deployment/<deployment-name>
```

- ✓ Ensure deployment has successfully rolled out all replicas.

9. Common Quick Fixes

- Update image tag or fix broken image
- Correct environment variables
- Mount missing Secrets / ConfigMaps

- Increase CPU / memory limits
- Fix permissions or file paths inside container

10. Document & Repeat

- Note the root cause and solution for future reference.

Keep a template checklist handy for repeated troubleshooting.

Scenario: Jenkins job is failing after a pipeline change. How will you troubleshoot?

Here's a **ready-to-use Jenkins Pipeline Troubleshooting Checklist** you can follow step by step whenever a Jenkins job fails. It's structured for quick reference.

Jenkins Pipeline Troubleshooting Checklist

1. Check Job Console Output

Go to Job → Build History → Failed Build → Console Output

- ✓ Look for:
 - Compilation errors
 - Test failures
 - Missing environment variables
 - Pipeline syntax errors
- ✓ Note the **exact error messages** for debugging.

2. Review Recent Pipeline Changes

- Compare the **current Jenkinsfile** / **shared library** with the last successful version.
- Look for:
 - Syntax or indentation errors
 - Stage/step modifications
 - Environment variable or credentials changes

3. Validate Jenkinsfile Syntax

- Use Jenkins **Pipeline Syntax Validator** or Jenkinsfile Linter Plugin.
- Ensure all steps and stages are correctly defined.

4. Check Agent / Node Environment

- Ensure the build node has all required tools installed:
 - JDK, Maven, Gradle, Node, Docker, etc.
- Check if the agent has sufficient CPU/memory.
- For Kubernetes agents:

```
kubectl logs <agent-pod>
```

5. Verify Credentials & Secrets

- Confirm credentials exist in Jenkins and are correctly referenced:
 - Username/password
 - SSH keys
 - API tokens
- Ensure environment variables or secret bindings match the pipeline.

6. Compare With a Working Build

- Identify differences between the failing and last successful build:
 - Workspace files
 - Environment variables
 - Pipeline syntax / shared library versions

7. Check Plugin Versions

- Some failures are caused by incompatible or updated plugins.
- Verify required plugins are installed and compatible with Jenkins version.

8. Clean Workspace & Retry

- Stale files can cause failures:

Job → Workspace → Delete Workspace → Re-run Pipeline

- Retry the build after cleaning.

9. Enable Debugging

- Add debug flags in the pipeline:

```
sh 'set -x'          // for shell commands
sh 'env'             // print environment variables
```

- This helps trace command execution and environment issues.

10. Test Stages Locally

- Run individual commands outside Jenkins:
 - Shell scripts, build commands, test suites
- Helps identify issues not caused by Jenkins itself.

11. Check Rollback / Recovery

- If pipeline modifies deployments or production systems:
 - Verify rollback steps or staging tests
 - Ensure previous state can be restored if needed

12. Document & Repeat

- Note root cause and solution for future reference.
- Maintain a **versioned Jenkinsfile** history.

✓ **Tip:** Keep a “last known working pipeline” commit handy to quickly diff changes when failures occur.

Scenario: Your EC2 instances is not accessible via SSH. What do you do?

EC2 SSH Troubleshooting Checklist

1. Verify Basic Connectivity

```
ping <EC2-public-IP>
```

- ✓ Ensure the instance responds (if ICMP is allowed).
- ✓ If ping fails:
 - Check instance public IP
 - Check network routing (VPC, subnet, route table)

2. Check Security Group Rules

- Go to **AWS Console** → **EC2** → **Security Groups** → **Inbound Rules**
- ✓ Ensure port **22** is allowed
- ✓ Source IP matches your client's IP (or 0.0.0.0/0 for testing)
- ✓ Outbound rules allow responses

3. Verify Network ACLs

- Go to **VPC** → **Subnets** → **Network ACLs**
- ✓ Ensure inbound/outbound rules allow port 22
- ✓ No deny rules blocking SSH

4. Confirm EC2 Instance State

- Go to **EC2** → **Instances**
- ✓ Instance is **running**
- ✓ System and Instance status checks passed
- ✗ Failed checks indicate OS, boot, or hardware issues

5. Check Key Pair and Username

```
ssh -i <key.pem> ec2-user@<EC2-public-IP>
```

- ✓ Ensure correct **private key** is used
- ✓ Use correct username based on AMI:
 - Amazon Linux → `ec2-user`

- Ubuntu → ubuntu
- RHEL → ec2-user or root

6. Verify Public IP / Network Routing

- ✓ Public instance has a **public IP** or Elastic IP
- ✓ Private instance requires **bastion host or VPN**
- ✓ Route table has Internet Gateway for public subnets

7. Check System Logs via AWS Console

- Go to **EC2** → **Instance** → **Actions** → **Get System Log**
- Look for:
 - Boot errors
 - SSH daemon (sshd) errors
 - Disk full or permission issues

8. Test from Another Network

- ✓ Try connecting from a different machine or network to rule out local firewall/ISP issues

9. Use EC2 Serial Console (if supported)

- ✓ Reset SSH config `/etc/ssh/sshd_config`
- ✓ Check firewall (iptables/firewalld)
- ✓ Restart SSH daemon

10. Recover via Temporary Instance

1. Stop the instance
2. Detach root volume
3. Attach volume to another working instance
4. Fix SSH configuration, permissions, or firewall
5. Reattach volume and start instance

11. Enable Alternative Access for Future

- ✓ Enable **AWS SSM Session Manager** for SSH-free access
- ✓ Backup key pairs and note instance usernames

✓ **Tip:** Always document changes to security groups, firewall, and key pairs to simplify future troubleshooting.

Scenario: A Terraform apply failed. What next?

Terraform Apply Failure Troubleshooting Checklist

1. Examine the Error Message

```
terraform apply
```

- ✓ Identify the type of error:
 - Syntax / configuration error
 - Provider / API error
 - Resource conflict
 - Permission / IAM issue
 - Quota or limit exceeded

2. Check Terraform Plan

```
terraform plan
```

- ✓ Review what Terraform intends to create/change
- ✓ Detect misconfigurations or missing variables

3. Validate Configuration

```
terraform validate
```

- ✓ Ensures Terraform files are **syntactically correct and structurally valid**

4. Review Variable Values

- Ensure all **required variables** are provided:

```
terraform plan -var 'variable_name=value'
```

- ✓ Check `terraform.tfvars` and environment variables

5. Inspect State File

```
terraform show  
terraform state list
```

- ✓ Look for **corrupted or out-of-sync state**
- ✓ Ensure resources are tracked correctly

6. Check Dependencies

- ✓ Ensure dependent resources exist and are correctly referenced
- ✓ Look for resource conflicts or missing modules

7. Refresh State

```
terraform refresh
terraform apply
```

- ✓ Sync Terraform state with real infrastructure

8. Check Provider / API Limits

- ✓ Ensure your user/service account has **proper IAM permissions**
- ✓ Verify quotas or limits aren't exceeded

9. Resolve Lock File Conflicts

- ✓ Terraform may lock state for concurrency safety

```
terraform force-unlock <LOCK_ID>
```

10. Apply in Small Steps

- ✓ Apply modules incrementally to isolate failing resources
- ✓ Use:

```
terraform apply -target=<resource_name>
```

11. Enable Debug Logging

```
TF_LOG=DEBUG terraform apply
```

- ✓ Inspect detailed logs for API failures or resource errors

12. Rollback / Cleanup Partial Resources

```
terraform destroy -target=<resource_name>
```

- ✓ Avoid leaving orphaned resources incurring cost

13. Document & Repeat

- ✓ Note the root cause and solution for future reference
- ✓ Keep versioned state backups and use remote backends (S3, GCS, Terraform Cloud)

✓ **Tip:** Always **plan first**, then apply. Use **remote backend** and **locking** for team safety.

Scenario: Docker container keeps restarting. How to debug?

Docker Container Crash Troubleshooting Checklist

1. Check Container Status

```
docker ps -a
```

- ✓ Look at the `STATUS` column:
 - Exited (code) → container crashed
 - Restarting → container is in a loop
- ✓ Note **exit code**:

```
docker inspect <container-id> --format='{{.State.ExitCode}}'
```

2. Inspect Container Logs

```
docker logs <container-id>
```

- ✓ Look for:
 - Application errors / exceptions
 - Misconfigurations
 - Missing files or dependencies
- ✓ For rapidly restarting containers:

```
docker logs --since 1m <container-id>
```

3. Run Container Interactively

```
docker run -it --entrypoint /bin/sh <image-name>
```

- ✓ Check:
 - Configuration files
 - Environment variables
 - File permissions
 - Network connectivity
 - Dependency availability

4. Check Docker Events

```
docker events
```

- ✓ Monitor real-time events for restarts or failures

5. Inspect Container Configuration

```
docker inspect <container-id>
```

- ✓ Check:
 - ENTRYPOINT / CMD

- Environment variables
 - Volume mounts
 - Restart policy
- ✓ Common issues:
 - Misconfigured `ENTRYPOINT`
 - Missing required env variables
 - Incorrect volume mounts

6. Check Resource Constraints

```
docker stats
```

- ✓ Ensure CPU/memory limits are sufficient
- ✓ Check host resources

7. Test Image Health

- Pull a fresh image:

```
docker pull <image-name>
docker run <image-name>
```

- ✓ Verify if problem persists with a clean container

8. Temporarily Disable Restart Policy

- To debug without restart loops:

```
docker update --restart=no <container-id>
docker start -ai <container-id>
```

- ✓ Allows you to view logs and interact before the container exits

9. Check Host System Logs

```
journalctl -u docker
```

- ✓ Look for Docker daemon or host-level issues causing crashes

10. Document & Repeat

- ✓ Record the root cause and fix for future reference
- ✓ Maintain environment variable, volume, and network documentation

✓ **Tip:** Start with **logs + exit code + interactive shell**—most container crashes can be diagnosed quickly using these steps.

Scenario: You see a CVE in image scan via Trivy/SonarQube. What actions do you take?

Docker Image CVE Remediation Checklist

1. Analyze the CVE

- ✓ Identify **CVE ID** and **severity** (Critical, High, Medium, Low)
- ✓ Note **affected package/library** and **version**
- ✓ Review CVSS score and security advisory

2. Verify Exposure

- ✓ Determine if the vulnerable component is **actually used** in the application
- ✓ Prioritize CVEs based on:
 - Network exposure
 - Privilege level
 - Runtime usage

3. Check for Fixes / Updates

- ✓ Look for **upgraded versions** of the package that resolve the CVE
- ✓ Check **official base image updates**

4. Update the Docker Image

- ✓ Update base image or package in your Dockerfile:

```
FROM ubuntu:latest
RUN apt-get update && apt-get upgrade -y <vulnerable-package>
```

- ✓ Rebuild the image:

```
docker build -t <image-name>:<tag> .
```

- ✓ Rescan the image to ensure CVE is resolved:

```
trivy image <image-name>:<tag>
```

5. Mitigation if Fix Not Available

- ✓ Apply **workarounds** or configuration changes to reduce risk
- ✓ Restrict access to vulnerable components via **network policies**
- ✓ Use **runtime security policies** (AppArmor, seccomp, PodSecurityPolicies)

6. Document and Communicate

- ✓ Record:
 - CVE ID

- Severity
- Remediation steps taken
- ✓ Notify **security/compliance teams** for critical vulnerabilities

7. Retest and Verify

- ✓ Rescan the rebuilt image to confirm the CVE is gone
- ✓ Validate CI/CD pipeline blocks vulnerable images from deployment

8. Continuous Monitoring

- ✓ Integrate **regular Trivy or other image scans** in your CI/CD pipeline
- ✓ Subscribe to **CVE notifications** for base images and libraries
- ✓ Track and patch new vulnerabilities in future maintenance cycles

✓ Tip:

- Critical CVEs in production should **trigger immediate patching**.
- Medium or low-severity CVEs can be **tracked for the next maintenance window**.

Scenario: How would you roll back a Kubernetes deployment?

Kubernetes Deployment Rollback Checklist

1. Check Deployment History

```
kubectl rollout history deployment/<deployment-name>
```

- ✓ List all revisions of the deployment
- ✓ Note the revision you want to roll back to

2. Roll Back Deployment

- Roll back to **previous revision**:

```
kubectl rollout undo deployment/<deployment-name>
```

- Roll back to a **specific revision**:

```
kubectl rollout undo deployment/<deployment-name> --to-revision=<revision-number>
```

- ✓ Kubernetes will update pods to match the selected revision

3. Monitor Rollout Status

```
kubectl rollout status deployment/<deployment-name>
```

- ✓ Wait for successfully rolled out message

- ✓ Ensure pods are running and ready

4. Verify Deployment

- ✓ Check deployment details:

```
kubectl get deployment <deployment-name> -o wide  
kubectl describe deployment <deployment-name>
```

- ✓ Confirm pods are running expected image/configuration:

```
kubectl get pods -l app=<app-label>
```

5. Pause / Resume Deployment (Optional)

- Pause rollout for manual adjustments:

```
kubectl rollout pause deployment/<deployment-name>
```

- Resume rollout after changes:

```
kubectl rollout resume deployment/<deployment-name>
```

6. Check for Issues

- ✓ Inspect individual pods if rollback fails:

```
kubectl describe pod <pod-name>  
kubectl logs <pod-name>
```

- ✓ Look for crash loops, readiness probe failures, or config issues

7. Document & Communicate

- ✓ Record:
 - Rollback reason
 - Revision number
 - Any encountered issues
- ✓ Notify team or stakeholders about the rollback

8. Prevent Future Failures

- ✓ Use versioned deployment manifests
- ✓ Keep **readiness/liveness probes** correctly configured
- ✓ Implement automated CI/CD checks before deployment

✓ Tip:

- Always verify **pod readiness** before marking a rollback as complete. Maintain a **history of deployment manifests** for auditing and troubleshooting.

Scenario: Application is deployed but not accessible. What do you check?

Application Not Accessible Troubleshooting Checklist

1. Check Pod Status

```
kubectl get pods -n <namespace>
```

- ✓ Ensure pods are **Running**
- ✓ Look for **CrashLoopBackOff**, **Error**, or **Pending**

2. Inspect Pod Logs

```
kubectl logs <pod-name> -n <namespace>
```

- ✓ Look for:
 - Application startup errors
 - Missing dependencies
 - Configuration issues

3. Verify Deployment / ReplicaSet

```
kubectl get deployment -n <namespace>
```

```
kubectl describe deployment <deployment-name> -n <namespace>
```

- ✓ Check **replica count** matches expected pods
- ✓ Ensure deployment has rolled out successfully

4. Check Services

```
kubectl get svc -n <namespace>
```

- ✓ Confirm **Service type** (ClusterIP, NodePort, LoadBalancer)
- ✓ Verify **port mapping** and target port are correct

5. Check Ingress / Load Balancer

```
kubectl get ingress -n <namespace>
```

```
kubectl describe ingress <ingress-name> -n <namespace>
```

- ✓ Verify hostnames, backend service, TLS configuration
- ✓ Ensure Ingress controller is running

6. Test Network Connectivity

- ✓ From inside cluster:

```
kubectl exec -it <pod-name> -- curl http://<service-name>:<port>
```

- ✓ From outside cluster:

- NodePort: `curl http://<node-ip>:<node-port>`
- LoadBalancer / Ingress: `curl http://<external-ip>`

7. Verify ConfigMaps / Secrets

- ✓ Ensure the application has access to all **configurations and secrets**
- ✓ Check for missing or incorrect environment variables

8. Inspect Firewall / Security Groups

- ✓ Verify cloud firewall/security group rules allow access to required ports

9. Check Resource Usage

```
kubectl describe pod <pod-name> -n <namespace>
kubectl top pod <pod-name> -n <namespace>
```

- ✓ Ensure CPU/memory limits are sufficient
- ✓ Verify pods aren't being **OOMKilled**

10. Check DNS / Endpoint Resolution

- ✓ Test DNS resolution inside pod:

```
kubectl exec -it <pod-name> -- nslookup <service-name>
```

- ✓ Ensure network policies allow traffic

11. Verify Application Health Probes

- ✓ Check **readiness/liveness probes**
- ✓ Ensure the application listens on the correct port

12. Document & Communicate

- ✓ Record root cause, actions taken, and resolution
- ✓ Notify team or stakeholders if production impact exists

✓ **Tip:** Start troubleshooting from **pod logs and status**, then move outward to **service connectivity, ingress, and external access**.

Scenario: Jenkins is slow. How do you optimize?

Jenkins Performance Optimization Checklist

1. Check System Resource Utilization

```
top
free -h
df -h
```

- ✓ Ensure CPU and memory are sufficient for running builds
- ✓ Check disk space for workspaces, logs, and artifacts

2. Analyze Build Queue

- Go to **Jenkins Dashboard** → **Build Queue**
- ✓ Look for:
 - Many builds waiting → increase executors or add nodes
 - Blocked builds due to **locked resources**

3. Check Executor Configuration

- Go to **Manage Jenkins** → **Nodes and Executors**
- ✓ Increase number of **executors per node** if resources allow
- ✓ Distribute builds evenly across nodes

4. Inspect Job and Pipeline Configuration

- ✓ Identify **long-running or inefficient builds**
- ✓ Parallelize build steps if possible
- ✓ Cache dependencies (Maven, npm, pip) to avoid repeated downloads

5. Clean Up Workspace & Builds

- ✓ Delete old workspaces using **Workspace Cleanup Plugin**
- ✓ Remove old builds/artifacts
- ✓ Configure **Build Discarder** to automatically prune old builds

6. Review Plugins

- ✓ Remove or disable **unused plugins**
- ✓ Update plugins to latest stable versions
- ✓ Monitor plugin performance for bottlenecks

7. Use Faster Storage

- ✓ Prefer **SSD disks** over HDD
- ✓ Avoid slow network storage for workspaces and logs

8. Enable Distributed Builds

- ✓ Use **master for orchestration only**
- ✓ Delegate builds to **agents/slaves**
- ✓ Scale build nodes or use **Kubernetes/Cloud agents**

9. Monitor Jenkins Logs and JVM

```
tail -f /var/log/jenkins/jenkins.log
```

- ✓ Look for repeated errors or slow plugin execution
- ✓ Monitor JVM heap usage; increase `JAVA_OPTS` memory if needed

10. Optimize Pipeline Design

- ✓ Use **Declarative Pipelines**
- ✓ Avoid unnecessary `checkout` or build steps
- ✓ Parallelize independent stages

11. Enable Caching

- ✓ Dependency caching for Maven, Gradle, npm, pip
- ✓ Docker image layer caching for containerized builds

12. Consider Jenkins Upgrade

- ✓ Upgrade to **latest stable LTS version**
- ✓ Performance improvements and bug fixes often included in new releases

✓ Tip:

- Use **Monitoring Plugin**, **Prometheus + Grafana**, or **Jenkins Performance Plugin** to continuously track performance metrics.

Prioritize **resource bottlenecks**, then optimize pipelines and plugins.

Scenario: How do you ensure zero-downtime deployment in Kubernetes?

Zero-Downtime Deployment Checklist for Kubernetes

1. Use Deployment Objects

- ✓ Ensure application is deployed via **Deployment** instead of directly managing pods
- ✓ Confirm **replicas** count is sufficient to handle traffic

2. Configure Rolling Update Strategy

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

- ✓ `maxUnavailable: 0` → ensures no pod is removed before a new one is ready
- ✓ `maxSurge: 1` → new pod created before old pod is terminated

3. Configure Readiness Probes

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
```

- ✓ Ensures traffic is only routed to **ready pods**
- ✓ Prevents serving requests to pods still starting

4. Configure Liveness Probes

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
```

- ✓ Automatically restart unhealthy pods
- ✓ Avoids downtime due to crashed pods

5. Use Kubernetes Services

- ✓ Deploy pods behind a **Service**
- ✓ Ensures **load balancing** across all ready pods
- ✓ Maintains stable endpoint for clients

6. Ensure Sufficient Replicas / Autoscaling

- ✓ Use **Horizontal Pod Autoscaler** if needed:

```
kubectl autoscale deployment my-app --cpu-percent=50 --min=3 --max=10
```

- ✓ Handles traffic spikes during deployment

7. Monitor Rollout

```
kubectl rollout status deployment/<deployment-name>  
kubectl get pods -l app=<app-label>  
kubectl logs <pod-name>
```

- ✓ Watch for pod readiness and errors
- ✓ Ensure all pods are healthy

8. Configure Graceful Shutdown

- ✓ Set `terminationGracePeriodSeconds` to allow connections to finish
- ✓ Avoid killing pods abruptly during rolling updates

9. Use Canary or Blue-Green Deployment (Optional for Critical Apps)

- ✓ Canary → deploy to subset first, monitor, then increase traffic
- ✓ Blue-Green → deploy new version alongside old, switch traffic when healthy

10. Plan for Rollback

```
kubectl rollout undo deployment/<deployment-name>
```

- ✓ Keep previous version available for instant rollback
- ✓ Minimize impact of unexpected issues

11. Automate Deployment via CI/CD

- ✓ Use ArgoCD, FluxCD, or Jenkins pipelines
- ✓ Integrate automated health checks and rollback triggers

12. Document & Communicate

- ✓ Record deployment details, version, and health check results
- ✓ Notify team/stakeholders about deployment completion

✓ Tip:

- Proper **readiness probes** + **rolling updates** + **service load balancing** are critical for zero-downtime deployments.

Scenario: Git merge conflict in CI/CD pipeline. What do you do?

Git Merge Conflict Resolution Checklist for CI/CD

1. Identify the Conflict

- ✓ Check pipeline logs or Git status:

```
git status
```

- ✓ Note the files causing conflicts

2. Stop the Pipeline (Optional)

- ✓ Prevent further deployments until conflicts are resolved

3. Fetch Latest Changes

```
git fetch origin
git checkout <branch>
git pull origin <branch>
```

- ✓ Ensure your branch is up to date with remote

4. Attempt Merge Locally

```
git merge <other-branch>
```

- ✓ Identify conflicting files
- ✓ Conflicts are marked in files with:

```
<<<<<<< HEAD
<your changes>
=====
<other branch changes>
>>>>>>> branch-name
```

5. Resolve Conflicts

- ✓ Open files and manually resolve conflicts
- ✓ Keep desired changes or combine logic correctly
- ✓ Remove conflict markers (<<<<<<<, =====, >>>>>>>)

6. Test Changes Locally

- ✓ Build the project
- ✓ Run all tests to ensure nothing is broken after resolving conflicts

7. Stage and Commit Resolved Files

```
git add <resolved-files>
git commit -m "Resolved merge conflict with <branch-name>"
```

- ✓ Ensure all resolved files are included in the commit

8. Push Changes

```
git push origin <branch>
```

- ✓ Pipeline can now resume with a clean merge

9. Trigger CI/CD Pipeline

- ✓ Verify the conflict resolution did not break the build
- ✓ Confirm successful completion of tests and deployment stages

10. Document & Communicate

- ✓ Record which files caused conflicts and how they were resolved
- ✓ Notify team members if multiple contributors are affected
- ✓ Encourage **smaller, frequent merges** to reduce future conflicts

✓ Tip:

- Use **feature branches and pull requests** to reduce direct conflicts on the main branch.

Consider **code ownership, clear responsibilities, and pre-merge pipeline checks** to minimize conflicts in critical areas.

Scenario: Your Terraform state file is corrupted. What can you do?

Terraform State Corruption Recovery Checklist

1. Identify the Issue

- ✓ Terraform commands fail with errors like `Failed to load state: corrupted state file`
- ✓ Confirm whether the corrupted state is **local** or **remote**

2. Backup Current State File

```
cp terraform.tfstate terraform.tfstate.backup
```

- ✓ Always create a **manual backup** before attempting recovery
- ✓ For remote backends, download a copy to a safe location

3. Check Remote State Versions

- ✓ Retrieve previous working versions from backend:
 - **S3:** Use versioning to restore older state
 - **Terraform Cloud:** Use **State Versions** tab
 - **GCS / Azure Storage:** Retrieve previous version from storage

4. Restore Previous Working State

```
cp terraform.tfstate.previous terraform.tfstate
```

- ✓ Use `terraform state push` if using remote backends
- ✓ Ensure restored state matches the expected infrastructure

5. Inspect and Repair State

```
terraform state list
terraform state show <resource>
```

- ✓ Identify problematic resources
- ✓ Remove corrupt resources if necessary:

```
terraform state rm <resource>
```

- ✓ Re-import resources to state if needed:

```
terraform import <resource> <id>
```

6. Validate Terraform Configuration

```
terraform validate
terraform plan
```

- ✓ Ensure configuration matches restored state

- ✓ Check for drift between state and actual infrastructure

7. Apply Changes Carefully

```
terraform apply
```

- ✓ Apply changes incrementally using `-target=<resource>` if necessary
- ✓ Monitor plan output to prevent unintended destruction

8. Enable State Locking & Versioning

- ✓ Use **remote backends with state locking**
- ✓ Enable **versioning** to allow easy rollback

9. Document & Communicate

- ✓ Record root cause, steps taken, and outcome
- ✓ Notify team about state recovery actions

✓ Tip:

- **Never manually edit** the state file unless as a last resort.
- Always use **remote backends with locking and versioning** in production environments to minimize risk of state corruption.

Scenario: ArgoCD sync fails. What's your debugging approach?

ArgoCD Sync Failure Troubleshooting Checklist

1. Check Application Status

```
argocd app get <app-name>
```

- ✓ Look at:
 - Sync Status: OutOfSync
 - Health Status: Progressing / Degraded / Unknown
 - Error messages in MESSAGE section

2. Inspect Sync Operation

```
argocd app history <app-name>  
argocd app logs <app-name>
```

- ✓ Review recent sync attempts
- ✓ Identify conflicts, missing resources, or permission errors

3. Check Kubernetes Resources

```
kubectl get all -n <app-namespace>  
kubectl describe <resource-type> <resource-name> -n <app-namespace>  
kubectl logs <pod-name> -n <app-namespace>
```

- ✓ Ensure all resources exist and are healthy
- ✓ Look for pods in CrashLoopBackOff or Pending

4. Inspect ArgoCD Controller Logs

```
kubectl logs -n argocd deploy/argocd-application-controller
```

- ✓ Look for:
 - Resource conflicts
 - Permission denied errors
 - Validation errors

5. Validate Manifests

- ✓ Run a **dry-run** to check syntax and API compatibility:

```
kubectl apply --dry-run=client -f <manifest>
```

- ✓ Confirm manifests match Kubernetes cluster API versions

6. Check Git Repository Access

- ✓ Ensure ArgoCD can access the repository:
 - Correct SSH key, token, or HTTPS credentials

- Branch/tag exists and matches ArgoCD configuration

7. Check RBAC and Permissions

- ✓ Ensure ArgoCD service account has privileges to create/update resources in the target namespace

8. Review Application Parameters

- ✓ Verify Helm chart values or Kustomize overlays
- ✓ Confirm referenced **secrets/configMaps** exist

9. Retry or Resync

```
argocd app sync <app-name>
```

- ✓ Attempt manual sync after resolving errors
- ✓ Monitor rollout and check for healthy resources

10. Document & Communicate

- ✓ Record root cause, resolution steps, and final outcome
- ✓ Notify team if production services were affected

✓ Tip:

- Start troubleshooting with **argocd app get** and **controller logs**; most sync failures are caused by **resource conflicts, permission issues, or manifest errors**.

Maintain a **versioned Git repository** to reduce configuration drift.

Scenario: Team needs to access secrets securely. How do you manage it?

Here's a **structured approach** to manage secrets securely for a team, especially in cloud-native and DevOps environments:

1. Use a Secret Management Tool

- ✓ Use dedicated secret management solutions rather than storing secrets in code or Git:
 - **HashiCorp Vault**
 - **AWS Secrets Manager**
- ✓ Ensure secrets are **encrypted at rest** and **in transit**

2. Define Access Control

- ✓ Use **role-based access control (RBAC)** to limit access to only those who need it
- ✓ Define **least privilege policies** for applications and users

3. Use Dynamic Secrets (Optional)

- ✓ For databases, cloud APIs, or ephemeral resources, use **dynamic secrets** that:
 - Auto-rotate
 - Expire after a defined time

4. Integrate Secrets with CI/CD

- ✓ Use secret injection into pipelines instead of hardcoding:
 - **Jenkins Credentials Plugin**
 - **GitHub Actions Secrets**
- ✓ Ensure secrets are **masked in logs**

5. Integrate Secrets with Kubernetes

- ✓ Use **Kubernetes Secrets** to store sensitive data:

```
kubectl create secret generic my-secret --from-literal=KEY=value -n <namespace>
```

- ✓ Mount secrets as **environment variables** or **volumes**
- ✓ Consider **SealedSecrets (Bitnami)** or **External Secrets Operator** to integrate cloud secrets

6. Enable Secret Rotation

- ✓ Rotate secrets periodically or automatically
- ✓ Use tools that support **automatic key rotation** to reduce exposure

7. Audit and Monitor Access

- ✓ Enable logging and auditing of secret access
- ✓ Detect unauthorized access or suspicious activity

8. Avoid Common Mistakes

- ✗ Never commit secrets to Git or code repositories
- ✗ Avoid plaintext storage in config files
- ✗ Avoid sharing secrets via chat/email

9. Educate the Team

- ✓ Train developers and ops on secure secret handling
- ✓ Enforce policies for **secret creation, rotation, and usage**

✓ Tip:

- Combine **RBAC + Secret Management + CI/CD integration** to reduce human errors

Consider **vault sidecar patterns** or **Kubernetes External Secrets** to inject secrets at runtime without storing them in code

Scenario: Application performance is degraded. Where do you start?

Application Performance Degradation Troubleshooting Checklist

1. Confirm the Symptoms

- ✓ Identify affected endpoints, services, or users
- ✓ Collect metrics: response times, error rates, throughput
- ✓ Determine whether degradation is **system-wide or isolated**

2. Check Application Logs

```
kubectl logs <pod-name> -n <namespace>
docker logs <container-name>
```

- ✓ Look for errors, exceptions, or warnings
- ✓ Identify slow queries, retries, or timeout issues

3. Monitor Resource Utilization

- ✓ CPU, memory, disk I/O, and network usage:

```
top
kubectl top pod -n <namespace>
docker stats
```

- ✓ Identify resource bottlenecks

- ✓ Check for OOMKilled pods, throttling, or high load

4. Check Dependencies

- ✓ Database:
 - Slow queries, locks, or connection pool exhaustion
- ✓ External APIs or services:
 - Latency spikes or failures
- ✓ Cache layers (Redis, Memcached)
 - Hit/miss ratio, latency

5. Check Networking

- ✓ Latency between services
- ✓ Network policies and DNS resolution
- ✓ Load balancer health and traffic distribution

6. Review Recent Changes

- ✓ Deployments, configuration, or infrastructure changes
- ✓ Compare performance before/after changes
- ✓ Consider rolling back recent changes if degradation coincides

7. Profiling & Tracing

- ✓ Use APM tools or profilers:
 - Jaeger, Zipkin, Prometheus + Grafana, NewRelic, Datadog
- ✓ Identify slow functions, queries, or I/O bottlenecks

8. Check Scaling and Load

- ✓ Ensure sufficient application replicas/resources
- ✓ Verify Horizontal Pod Autoscaler or cluster scaling
- ✓ Analyze traffic spikes or unusual load patterns

9. Collect Metrics & Alerts

- ✓ CPU, memory, disk, and network usage over time
- ✓ Application-specific metrics: request latency, error rate
- ✓ Identify patterns of degradation or peak times

10. Document & Communicate

- ✓ Record root cause, actions taken, and resolution steps
- ✓ Notify stakeholders if production impact is significant

✓ **Tip:**

- Start troubleshooting from **logs + resource monitoring + dependencies**
- Focus on **bottlenecks**: CPU/memory, database, cache, external APIs, or network

Use metrics and tracing to identify **root cause** before making changes

Scenario: Dockerfile is slow during build. How to optimize?

Dockerfile Build Optimization Checklist

1. Analyze the Dockerfile

- ✓ Identify long-running steps (RUN, COPY, ADD)
- ✓ Check for unnecessary commands or repeated steps
- ✓ Locate large file copies that could be optimized

2. Use Efficient Base Images

- ✓ Prefer **slim or minimal images** (e.g., python:3.12-slim, alpine)
- ✓ Avoid installing unnecessary packages in the base image

3. Optimize Layer Caching

- ✓ Reorder Dockerfile instructions:
 - Rarely changed layers **first**
 - Frequently changed layers **later**
- ✓ Leverage Docker cache for faster rebuilds

4. Minimize File Copy

- ✓ Use `.dockerignore` to exclude unnecessary files
- ✓ Avoid copying large directories unless required for the build

5. Combine Commands

- ✓ Merge multiple RUN commands to reduce layers:

```
RUN apt-get update && apt-get install -y \  
    package1 package2 && \  
    rm -rf /var/lib/apt/lists/*
```

- ✓ Reduces build time and final image size

6. Use Multi-Stage Builds

- ✓ Separate **build environment** from **runtime environment**
- ✓ Copy only necessary artifacts to final image:

```
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

7. Cache Dependencies

- ✓ Cache package manager dependencies (npm, pip, apt)
- ✓ Use **Docker BuildKit** for improved caching:

```
DOCKER_BUILDKIT=1 docker build .
```

8. Reduce Image Size

- ✓ Remove temporary files and caches after installation
- ✓ Use minimal base images in the final stage

9. Parallelize Dependency Installation

- ✓ Use parallel options in package managers (npm, pip, Maven) to speed up installation

10. Monitor Build Time

- ✓ Track duration of each Dockerfile step:

```
docker build --progress=plain .
```

- ✓ Identify slow steps for further optimization

✓ Tip:

- Focus first on **layer caching, multi-stage builds, and minimizing file copies**

Use **lightweight base images** and **clean up temporary files** to reduce build time and image size

Scenario: You accidentally deleted a Kubernetes namespace. Recovery plan?

Kubernetes Namespace Deletion Recovery Checklist

1. Confirm Namespace Deletion

```
kubectl get namespaces
```

- ✓ Verify that the namespace is missing or stuck terminating
- ✓ Identify all affected resources

2. Check for Finalizers / Stuck Terminating

```
kubectl get namespace <namespace> -o json
```

- ✓ Look for `finalizers` that may block deletion
- ✓ If stuck, remove finalizers carefully:

```
kubectl patch namespace <namespace> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

3. Restore from Backups

- ✓ Check for **etcd snapshots** or cluster backups
- ✓ Use backup tools like **Velero**:

```
velero restore create --from-backup <backup-name>
```

- ✓ Restore manifests, resources, and persistent volumes

4. Recreate Namespace and Resources

- ✓ Recreate namespace:

```
kubectl create namespace <namespace>
```

- ✓ Reapply manifests, Helm releases, or Kustomize overlays

5. Restore Persistent Volumes / Data

- ✓ Reattach or restore **Persistent Volumes (PV)** and **PersistentVolumeClaims (PVC)**
- ✓ Restore database or application-specific backups

6. Validate Resources

```
kubectl get all -n <namespace>
```

- ✓ Ensure deployments, pods, services, configmaps, and secrets are restored
- ✓ Confirm pods reach `Running` and services are accessible

7. Update CI/CD & ArgoCD Pipelines

- ✓ Ensure pipelines point to the restored namespace
- ✓ Resync ArgoCD or redeploy Helm releases

8. Communicate & Document

- ✓ Notify the team about the incident
- ✓ Record root cause, recovery steps, and preventive measures

9. Prevent Future Accidents

- ✓ Enable **RBAC restrictions** on namespace deletion
- ✓ Use **admission controllers** to prevent accidental deletions
- ✓ Maintain **regular backups** of namespaces and resources

✓ Tip:

- Always maintain **namespace-level backups** and **versioned manifests** for quick recovery.

Using **Velero** or **cloud-native backup solutions** automates disaster recovery and reduces manual intervention

Scenario: Jenkins pipeline takes too long. How can you speed it up?

Jenkins Pipeline Performance Optimization Checklist

1. Analyze Pipeline Execution

- ✓ Use **pipeline logs and stage view** to identify slow stages
- ✓ Measure duration per step with timestamps
- ✓ Record recurring bottlenecks for future optimization

2. Optimize Build Environment

- ✓ Use dedicated agents with **sufficient CPU, memory, and disk I/O**
- ✓ Use **Docker agents** for isolated and consistent build environments
- ✓ Scale agents horizontally for **parallel builds**

3. Cache Dependencies

- ✓ Cache Maven, npm, pip, or other dependencies between builds
- ✓ Use Jenkins caching plugins or **stash/unstash** mechanism

- ✓ Store dependencies in **artifact repositories** (Nexus, Artifactory)
- ✓ Avoid re-downloading dependencies for every build

4. Parallelize Independent Stages

- ✓ Split independent stages into **parallel execution**:

```
parallel(  
  stage1: { /* steps */ },  
  stage2: { /* steps */ }  
)
```

- ✓ Reduces total pipeline runtime

5. Optimize Tests

- ✓ Run **unit tests** separately from **integration or end-to-end tests**
- ✓ Use **test impact analysis** to run only affected tests
- ✓ Cache test results to avoid redundant execution

6. Use Incremental Builds

- ✓ Build only **changed modules/components**
- ✓ Use **Maven incremental build**, **Gradle incremental tasks**, or **npm incremental builds**

7. Optimize Docker Builds (if applicable)

- ✓ Use **layer caching**, **multi-stage builds**, and **minimal base images**
- ✓ Prebuild base images to reduce build time in CI/CD

8. Optimize Artifact Uploads

- ✓ Compress artifacts before upload
- ✓ Use **fast storage** for artifact repositories
- ✓ Upload only required artifacts

9. Monitor and Profile Pipelines

- ✓ Use **Jenkins Pipeline Stage View** or **Blue Ocean** for visualization
- ✓ Track metrics over time to identify recurring bottlenecks
- ✓ Log pipeline performance data for trend analysis

10. Communicate & Document

- ✓ Record improvements and bottleneck patterns
- ✓ Share **best practices** with the team
- ✓ Maintain documentation on optimized pipeline patterns

✓ Tip:

- Start with **parallel execution, caching dependencies, and incremental builds** for maximum speed improvement.
- Scale Jenkins agents horizontally to handle multiple concurrent pipelines efficiently.

Regularly monitor pipeline performance to prevent regressions

Scenario: Your Kubernetes pod can't reach an external service. What now?

Here's a **structured approach** to troubleshoot a **Kubernetes pod that can't reach an external service**:

1. Confirm the Issue

- ✓ Exec into the pod:

```
kubectl exec -it <pod-name> -n <namespace> -- /bin/sh
```

- ✓ Test connectivity to the external service:

```
ping <external-service>
curl -v <external-service>
```

- ✓ Determine if the issue is **DNS, routing, or firewall related**

2. Check Pod DNS Resolution

```
nslookup <external-service>
```

- ✓ Confirm pod can resolve the hostname
- ✓ If DNS fails, check **CoreDNS pods**:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs <coredns-pod> -n kube-system
```

3. Check Network Policies

- ✓ Verify **NetworkPolicy** restrictions in the namespace:

```
kubectl get networkpolicy -n <namespace>
```

- ✓ Ensure outbound traffic to external IPs/ports is allowed

4. Check Pod Routing

- ✓ Verify default route in the pod:

```
ip route
```

- ✓ Check if pod can reach cluster **gateway or NAT**
- ✓ Test connectivity from another pod in the same namespace

5. Check Cluster Egress

- ✓ If using a cloud provider, confirm **NAT Gateway / Internet Gateway** for private nodes
- ✓ Ensure **firewalls or security groups** allow egress to external service

6. Verify Service Availability

- ✓ Test external service availability from **outside the cluster**
- ✓ Confirm the service endpoint, port, and protocol are correct

7. Review Pod Logs

```
kubectl logs <pod-name> -n <namespace>
```

- ✓ Look for errors related to **connection refused, timeout, or DNS resolution**

8. Test Alternative Tools

- ✓ Use temporary debug pods (e.g., nicolaka/netshoot) for advanced networking troubleshooting:

```
kubectl run -it --rm debug --image=nicolaka/netshoot -- /bin/bash
```

- ✓ Run `tracert`, `curl`, or `dig` from inside the pod

9. Document & Communicate

- ✓ Record findings, root cause, and resolution steps
- ✓ Notify team or network administrators if cluster egress needs adjustment

✓ Tip:

- Start with **DNS resolution and network policies**, as most pod-to-external connectivity issues stem from these areas.

Use debug tools like **netshoot**, **tcpdump**, or **tracert** for deeper network troubleshooting.

Scenario: How would you handle disaster recovery for your infrastructure?

Disaster Recovery Checklist for Infrastructure

1. Define Recovery Objectives

- ✓ Establish **Recovery Point Objective (RPO)** – max tolerable data loss
- ✓ Establish **Recovery Time Objective (RTO)** – max acceptable downtime
- ✓ Prioritize critical systems and applications

2. Inventory Infrastructure and Dependencies

- ✓ List all infrastructure components: VMs, containers, databases, storage, networking
- ✓ Document service dependencies and critical paths
- ✓ Identify **single points of failure**

3. Implement Backup Strategy

- ✓ Schedule regular backups for:
 - Databases (RDS, MySQL, MongoDB, etc.)
 - Persistent volumes and storage
 - Configuration files, manifests, IaC (Terraform, Helm charts)
- ✓ Store backups **offsite or in a different region**
- ✓ Automate backups and verify **backup integrity**

4. Use Infrastructure as Code

- ✓ Manage infrastructure via Terraform, CloudFormation, or ARM templates
- ✓ Store IaC in **versioned Git repositories**
- ✓ Enables rapid recreation of environments during recovery

5. Implement High Availability

- ✓ Deploy services across multiple **availability zones/regions**
- ✓ Use **load balancers, auto-scaling groups, and multi-region replication**
- ✓ Ensure databases support replication and failover

6. Establish Monitoring and Alerts

- ✓ Monitor system health with Prometheus, Grafana, CloudWatch, or Datadog
- ✓ Set alerts for:
 - System failures
 - Backup failures
 - High latency or error rates
- ✓ Ensure proactive detection to minimize downtime

7. Create Disaster Recovery Runbooks

- ✓ Document step-by-step recovery procedures:
 - Infrastructure restoration
 - Database restoration
 - Application redeployment
- ✓ Include contact info for responsible team members

8. Test Recovery Procedures

- ✓ Conduct **DR drills** in non-production environments
- ✓ Test both **partial service failures** and **full region outages**
- ✓ Validate **RPO and RTO targets**

9. Automate Recovery Where Possible

- ✓ Use scripts or IaC to quickly **recreate environments**
- ✓ Integrate recovery automation with CI/CD pipelines

10. Review and Update DR Plan

- ✓ Update DR plan whenever infrastructure or application changes occur
- ✓ Review at least **annually** or after an incident
- ✓ Keep team informed of updates and procedures

✓ Tip:

- Focus first on **critical services and data**, then extend DR procedures to less critical components.

Combine **backups, high availability, monitoring, automation, and testing** for a robust disaster recovery strategy